

Session 3: Introduction to R in Rstudio

Heiner Salomon

Bremen University

2026-05-04



Overview I

- 1 Introduction
- 2 Short repetition from last week
- 3 Short discussion of tasks
- 4 RStudio basics
- 5 Break
- 6 Tasks for the week
- 7 Any questions?

Introduction

Learning goals of today

- Short repetition from last week



- Short repetition from last week
- Talking about best coding practices
- Discussing research questions

Short repetition from last week

- Projects are useful “shells” for your work
- Make your work portable

- Four different windows

- Four different windows
 - Script (top left)

- Four different windows
 - Script (top left)
 - Temporary files (“Environment”, top right)

- 
- Universität
Bremen



- Please state your research question
- What data (and variables) do you want to use?

- Please state your research question
- What data (and variables) do you want to use?
- Current questions?

Universität
Bremen

- Today only basics (!)
- What is what in R
- R basics

- Today only basics (!)
- What is what in R
- R basics
- Packages and help
- Loading data

- © 2006 Blackwell Publishing Ltd





- keep it simple
- “The computer” is stupid, so spelling matters (!)

- 
- Universität
Bremen

Terminology: Data types in R?

- Numeric <- any variable with free numbers
 - eg population, meters etc.
- Factor <- specific number with specific meanings
 - eg sex, employment status etc.



Terminology: Data types in R?

- Numeric <- any variable with free numbers
 - eg population, meters etc.
- Factor <- specific number with specific meanings
 - eg sex, employment status etc.
- String <- any text, delineated by “ ”



Assignment

- Assigning an object to name to refer to it later

```
# Load a data set
iris_data <- iris
# Assign value 2 to x
x <- 2
```



Assignment

- Assigning an object to name to refer to it later
- Using the `<` followed by `-` like this `"<-"`

```
# Load a data set
iris_data <- iris
# Assign value 2 to x
x <- 2
```



Assignment

- Assigning an object to name to refer to it later
- Using the `<` followed by `-` like this `"<-"`
- Gets stored in top right under "Global environment"

```
# Load a data set
iris_data <- iris
# Assign value 2 to x
x <- 2
```



How to get to variables in Base R?

- We will mostly use tidyverse

```
# Speichern der Variablen Sepal.Length von iris in y  
y <- iris$Sepal.Length
```



How to get to variables in Base R?

- We will mostly use tidyverse
- still this seemed quite relevant

```
# Speichern der Variablen Sepal.Length von iris in y  
y <- iris$Sepal.Length
```



How to get to variables in Base R?

- We will mostly use tidyverse
- still this seemed quite relevant
- refer to next level of nesting using \$

```
# Speichern der Variablen Sepal.Length von iris in y  
y <- iris$Sepal.Length
```



How to get to variables in Base R?

- We will mostly use tidyverse
- still this seemed quite relevant
- refer to next level of nesting using \$
- is visualised through clickable errors in “Environment”

```
# Speichern der Variablen Sepal.Length von iris in y  
y <- iris$Sepal.Length
```



Commenting

- Comments are essential in understanding code

```
# This is a comment  
2+2
```

Commenting

- Comments are essential in understanding code
- Comments are added by starting line with #

```
# This is a comment  
2+2
```

Code sections

- You should try to code in sections

Code sections

- You should try to code in sections
- Like headers in text, makes it easier to follow code

Code sections

- You should try to code in sections
- Like headers in text, makes it easier to follow code
- Sections let you “jump” to code in outline and at bottom of script window

Code sections - how to

- Consist of comment `#` followed by section name followed by at least four `- - - -`

```
# This is a section ----  
2+2
```



Universität
Bremen

Code sections - how to

- Consist of comment `#` followed by section name followed by at least four `- - - -`
- Hidden (“folded”) using arrow on left next to line number

```
# This is a section ----  
2+2
```

Code sections - how to

- Consist of comment `#` followed by section name followed by at least four `- - - -`
- Hidden (“folded”) using arrow on left next to line number
- Un-hidden using the blue double-arrow next to section name

```
# This is a section ----  
2+2
```

Code sections - subsections

- Can be useful at times in longer files

```
# This is a section ----  
v <- 2  
## This is a sub-section ----  
w = v + 3
```



Code sections - subsections

- Can be useful at times in longer files
- You can use `##` or `###` for different subsections

```
# This is a section ----  
v <- 2  
## This is a sub-section ----  
w = v + 3
```

Packages

- R is built up by a whole ecosystem of “packages”

Packages

- R is built up by a whole ecosystem of “packages”
- packages are essentially a (set of) function(s) others wrote to simplify your life

Packages

- R is built up by a whole ecosystem of “packages”
- packages are essentially a (set of) function(s) others wrote to simplify your life
- “Under the hood” is the so-called Base-R

Packages

- R is built up by a whole ecosystem of “packages”
- packages are essentially a (set of) function(s) others wrote to simplify your life
- “Under the hood” is the so-called Base-R
- But most people use packages, especially tidyverse



Packages - how to

- First package needs to be installed using
“install.packages(“”)”

```
# Install the tidyverse
install.packages("tidyverse")
install.packages("tidylog")
# Load the package
library(tidyverse)
library(tidylog)
```



Packages - how to

- First package needs to be installed using `install.packages("")`
- Then loaded using `library()`

```
# Install the tidyverse
install.packages("tidyverse")
install.packages("tidylog")
# Load the package
library(tidyverse)
library(tidylog)
```



Packages - life-hack

- usually, one loads all the necessary packages at the beginning of a script

```
# Using a package without loading it  
readr::read_csv("your_data.csv")
```



Universität
Bremen

Packages - life-hack

- usually, one loads all the necessary packages at the beginning of a script
- if you don't want to do that you can also use the following

```
# Using a package without loading it  
readr::read_csv("your_data.csv")
```



Universität
Bremen

Packages - life-hack

- usually, one loads all the necessary packages at the beginning of a script
- if you don't want to do that you can also use the following
- `packagename::functionname`

```
# Using a package without loading it  
readr::read_csv("your_data.csv")
```



Universität
Bremen

Help!

- if you need help with a function, type in ? followed by the function name

```
# Help me with the function summary()
?summary()
# Then you can use it
summary(iris)
```



Help!

- if you need help with a function, type in ? followed by the function name
- Many tidyverse functions have own website

```
# Help me with the function summary()
?summary()
# Then you can use it
summary(iris)
```



Help!

- if you need help with a function, type in ? followed by the function name
- Many tidyverse functions have own website
- alternative: throw your question in your search machine

```
# Help me with the function summary()
?summary()
# Then you can use it
summary(iris)
```



Loading data

- loading data into R using `load()`

Loading data

- loading data into R using `load()`
- often data not stored in Rdata file

Loading data

- loading data into R using `load()`
- often data not stored in Rdata file
- import from other file types necessary

Importing data - useful packages

- Excel (.xlsx) -> readxl::read_excel()

Importing data - useful packages

- Excel (.xlsx) -> readxl::read_excel()
- Delimited (.csv) -> readr::read_csv()

Importing data - useful packages

- Excel (.xlsx) -> readxl::read_excel()
- Delimited (.csv) -> readr::read_csv()
- Stata (.dta) -> haven::read_dta()

Importing data - useful packages

- Excel (.xlsx) -> readxl::read_excel()
- Delimited (.csv) -> readr::read_csv()
- Stata (.dta) -> haven::read_dta()
- SPSS (.sav) -> haven::read_sav()

Summarizing data

- It is crucial (!!!) to understand the data one is working with

Summarizing data

- It is crucial (!!!) to understand the data one is working with
- Many functions can be used to get overview

Summarizing data - View()

- `View(name_dataframe)` to see data

Summarizing data - View()

- View(name_dataframe) to see data
- Alternative to clicking on data set in “Environment” on top right

Summarizing data - View()

- View(name_dataframe) to see data
- Alternative to clicking on data set in “Environment” on top right
- opens up new tab with spreadsheet view on data

Summarizing data - View()

- View(name_dataframe) to see data
- Alternative to clicking on data set in “Environment” on top right
- opens up new tab with spreadsheet view on data
- can be very slow and unwieldy with large data sets

Summarizing data - head()

- `head(name_dataframe)` to see first five rows of data

Summarizing data - head()

- `head(name_dataframe)` to see first five rows of data
- good quick overview

Summarizing data - head()

- `head(name_dataframe)` to see first five rows of data
- good quick overview
- much faster than `View()`

Summarizing data - head()

- `head(name_dataframe)` to see first five rows of data
- good quick overview
- much faster than `View()`
- May not show data complexity

Summarizing data - names()

- `names(name_dataframe)` to see list of all variable names

Summarizing data - names()

- `names(name_dataframe)` to see list of all variable names
- can be quite useful

Summarizing data - names()

- `names(name_dataframe)` to see list of all variable names
- can be quite useful
- not always helpful in long data sets

Summarizing data - summary()

- `summary(name_dataframe)` to see distribution of variables

Summarizing data - summary()

- `summary(name_dataframe)` to see distribution of variables
- good quick overview

Summarizing data - summary()

- `summary(name_dataframe)` to see distribution of variables
- good quick overview
- limited usefulness, I personally find

- Universität

Summarizing data - skimr::skim()

- install skimr
- `library(skimr)` and `skim(name_dataframe)` or `skimr::skim()` to see data set

Break

Tasks for the week



- No specific tasks this week
- keep on working on your projects

Reading for next week

- Baker, D. (2013): How Much Unemployment Was Caused by Reinhart and Rogoff's Arithmetic Mistake?
<https://cepr.net/publications/how-much-unemployment-was-caused-by-reinhart-and-rogooffs-arithmetic-mistake/>

Reading for next week

- Baker, D. (2013): How Much Unemployment Was Caused by Reinhart and Rogoff's Arithmetic Mistake?
<https://cepr.net/publications/how-much-unemployment-was-caused-by-reinhart-and-rogooffs-arithmetic-mistake/>
- We will discuss those ideas at the beginning of next week's class

Any questions?